

Reliable Encounter Updates & AI Summaries

BACKEND ENGINEER / CLINICAL AI — Candidate exercise

Design a backend that stays correct when updates repeat, arrive out of order, or fail during processing.

The scenario

A partner system sends updates for a clinical encounter (a patient visit — e.g. an appointment, a medication refill request, or a telephone triage call). Your service must store the latest encounter state and generate an AI summary in the background. A client needs to check progress and retrieve the current summary.

Input contract

```
{
  "event_id": "evt-102",
  "encounter_id": "enc-42",
  "patient_id": "pat-77",
  "encounter_type": "TelephoneTriage",
  "version": 12,
  "payload": { "transcription": "Nurse: Thanks for calling, how can I help today? Patient: ..." }
}
```

- event_id is globally unique. Retries reuse the same ID and identical content.
- encounter_id identifies one encounter. version is a positive integer, increasing per encounter; distinct encounters have independent version sequences. Versions may be skipped (version 12 is usable even if version 11 never arrives).
- There is at most one logical event per (encounter_id, version). Treat this pair as unique even if a source defect sends the same logical update under a different event_id.
- patient_id identifies the patient the encounter belongs to. **It is fixed for a given encounter_id** — every event for the same encounter_id carries the same patient_id.
- encounter_type is one of Appointment, MedicationRefill, TelephoneTriage. Like patient_id, **it is fixed for a given encounter_id** — an encounter does not change type over its lifetime.
- payload.transcription is the full transcription of the encounter — the recorded back-and-forth between patient and nurse (or provider) — not free-text notes. It can be long (a multi-minute call or visit transcribed in full) and is passed as-is to generate_summary.
- Each payload is a complete snapshot, not an incremental change.
- Events may arrive more than once, concurrently, or out of order. For this exercise, the source does not send conflicting payloads for the same encounter version, and does not send conflicting patient_id/encounter_type for the same encounter_id — but your design should decide what to do if either invariant is ever violated (see failure scenarios).

Example payloads by encounter type

Transcriptions drive summary size and generate_summary latency, so treat them as long — potentially several hundred lines for a real call or visit. The excerpts below are synthetic and abbreviated; assume the full transcript continues in the same style.

Appointment (annual follow-up visit)

```
{
  "event_id": "evt-501",
  "encounter_id": "enc-201",
  "patient_id": "pat-33",
  "encounter_type": "Appointment",
  "version": 1,
  "payload": {
    "transcription": "Nurse: Good morning. How have you been feeling since your last visit? Patie
Pretty good overall, but a little more tired than usual. Nurse: Any changes in appetite or sleep?
Patient: No. Nurse: Blood pressure is 122 over 78 and heart rate is 72. Are you still taking lisinopril
daily? Patient: Yes. [transcript continues through physician exam, labs review, and care plan discussion
– truncated for this example]"
  }
}
```

```
}
```

MedicationRefill

```
{
  "event_id": "evt-502",
  "encounter_id": "enc-202",
  "patient_id": "pat-58",
  "encounter_type": "MedicationRefill",
  "version": 1,
  "payload": {
    "transcription": "Nurse: Hi, this is the refill line. How can I help? Patient: I need to refi
metformin; I'm almost out. Nurse: Are you still taking five hundred milligrams twice a day with meals?
Patient: Yes. Nurse: Any side effects? Patient: Not now, but my morning blood sugar has been a little
high. [transcript continues with medication review and pharmacy confirmation – truncated for this
example]"
  }
}
```

TelephoneTriage

```
{
  "event_id": "evt-503",
  "encounter_id": "enc-203",
  "patient_id": "pat-91",
  "encounter_type": "TelephoneTriage",
  "version": 1,
  "payload": {
    "transcription": "Nurse: Triage line, what's going on today? Patient: I've had a fever since
night and now my throat hurts. Nurse: Any difficulty breathing or swallowing? Patient: Swallowing is
painful, but breathing is fine. My child had strep throat last week. Nurse: Given your symptoms and
exposure, I'd like you to be seen today. [transcript continues with assessment and follow-up
instructions – truncated for this example]"
  }
}
```

Summary service contract

Assume the background worker calls:

```
generate_summary(transcription) -> summary
```

The call:

- typically takes **2-8 seconds**, but may occasionally take longer than 10 seconds;
- may time out or return transient errors, including rate-limit and service-availability errors;
- is **not guaranteed to be idempotent**; repeated calls may produce different wording; and
- incurs a cost on every invocation, including retries.

You do not need to design the summary-generation model itself. Focus on reliable orchestration around this external dependency.

Required behavior

- Accept newer encounter versions without letting older or repeated updates regress the stored state.
- A valid request may be acknowledged successfully even if it does not advance current state. The API response must distinguish an update that is **accepted and current**, a **duplicate**, and a **stale** update, and should report the current version where useful. State and justify your HTTP status-code choices.
- Do not wait for AI generation before acknowledging an accepted/current update. Accepted work must survive service restarts.
- Provide a way to retrieve summary progress and results. A summary must be associated with the exact encounter version it describes; an older result must never appear as current.
- You may retain immutable records for every encounter version, or retain only the current encounter plus an immutable processing snapshot for each accepted summary job. Explain your choice and guarantee that a worker can never summarize the wrong version.
- Obsolete summary work may be cancelled, marked as superseded, or allowed to complete and optionally retained for history. Regardless of that choice, an obsolete result must never become the current summary. Explain the cost and complexity trade-off.

- Store `patient_id` and `encounter_type` alongside encounter state, and make clear in your data model how they relate to `encounter_id` and to the summary record.
- Target an SLA of **under 10 seconds** from acceptance to a ready summary under normal AI service conditions. Since `generate_summary` may still take longer, time out, or fail, show how the system detects and handles an SLA breach. An SLA breach is operational information — for example, timestamps, a metric, alert, or derived `sla_breached` field — and is not by itself a workflow failure. A job may remain processing after breaching the SLA and become failed only according to the failure policy you define.

Your submission

Explain your decisions clearly. Choose any familiar language and database; pseudocode is sufficient. State additional assumptions and keep the design focused.

1 / Describe your design

- **API contract:** Show example requests and responses for accepting an update and retrieving its summary. Cover accepted/current, duplicate, stale, processing, ready, and failed outcomes.
- **Data model:** Sketch records, keys, constraints, and relationships. Show how `patient_id`, `encounter_type`, encounter versions, immutable summary inputs, work status, and results are represented. Include uniqueness or equivalent enforcement for both `event_id` and `(encounter_id, version)`.
- **Processing logic:** Provide pseudocode for ingestion and the background worker. Identify transaction boundaries, concurrency handling, stale-result protection, and retry behavior. Account for at-least-once execution and the non-idempotent, per-call cost of `generate_summary`.

2 / Walk through these failure scenarios

For each scenario, explain the expected stored state, what the client sees, and how processing continues or recovers.

Scenario	What happens?
Concurrent duplicate	Two service instances receive the same event concurrently.
Missing or late version	Version 12 arrives while version 10 is stored. Version 11 arrives later, or never arrives.
Crash after saving	The encounter update is saved, but the service crashes before handing work to the summary worker.
Worker interruption	A worker crashes during processing, or after saving a result but before acknowledging the job.
Newer version during processing	Version 12 is actively being summarized when version 13 arrives. What happens to version 12's work and result?
Results out of order	Version 13 finishes summarizing before version 12.
AI outage and retry exhaustion	The AI service is unavailable for 20 minutes. Explain backoff and retry behavior — including how it accounts for the per-call cost of retrying a non-idempotent, paid operation — what happens when the retry policy is exhausted, how failed work can be inspected or safely redriven, and what the client sees throughout.
Inconsistent identity fields	An event for an existing <code>encounter_id</code> arrives with a different <code>patient_id</code> or <code>encounter_type</code> than previously stored.

3 / Show how you would verify it

List 4–6 meaningful test cases with expected outcomes. Include a concurrency case and a crash/retry case. Briefly name the metrics, timestamps, alerts, or logs you would use to investigate a stuck or SLA-breaching summary without logging patient content.

Before submitting

Include your name, assumptions, and any known limitations. Return your document through the channel where you received this task. If you include optional code, attach it or link to it and provide run instructions. You should be able to explain every part of your submission.